F.1 Comparison to Bag-of-Tokens explanation

To control for the potential alternative explanation that the injected primitive vectors simply boost a "bag of tokens," increasing the probability of specific tokens, we also injected the primitive vector at the end of the residual stream, directly before the unembedding. We reasoned that if the primitive vector simply boosts a bag of tokens, the algorithmic expression should be evoked with similar strength by this procedure. We computed the maximal effect of this intervention across a wide range of magnitudes ($\alpha = 0.5, 1.0, 1.25, 1.5, 1.75, 2.0, 3.0, 4.0, 10.0, 100.0, 1000.0$) and subtracted it from the effect obtained by injecting the primitive vector into intermediate layers. We found that each primitive vector most strongly induced its corresponding behavioral hallmark even with this control, indicating that this behavior is caused by a latent procedure (Table 5).

	%NN paths ↑	#Paths ↑	Dist. comp. ↓	Final ans. ↓	#Verif. ↑	#Comp. ↑
`nearest_neighbor`	**+33.2%**	+47.3%	-42.2%	-18.6%	+9.8%	+141.5%
`generate_new_path`	+0.0%	**+73.9%**	-19.7%	-0.3%	-213.0%	-7.3%
`compute_distance`	+10.1%	+33.7%	**-44.8%**	-50.5%	+128.3%	+103.7%
`produce_final_answer`	+12.8%	+34.0%	-21.2%	**-63.5%**	+47.8%	-1.2%
`compare_or_verify`	+7.9%	+38.5%	-38.0%	-20.2%	**+559.8%**	**+1059.8%**
`compare_or_verify_2`	+5.9%	+0.8%	-24.3%	-24.7%	+516.3%	+290.2%

Table 5: Effects of primitive vector injection on behavioral hallmarks (% above baseline), controlling for the effect of boosting a "bag of tokens". For each cell, we identify the maximal effect across all positive magnitudes and all intervention layers (10, 13, 15, 17, 20, 30) and subtract the maximal effect across all positive magnitudes of injecting the primitive vector at the end of the residual stream. Bold = strongest effect, underline = second strongest per column. *(%NN paths: proportion of nearest neighbor paths generated. Dist. comp.: Distance computation. Final ans.: Final answer. #Verif.: number of Verifications in the output. #Comps: number of comparisons in the output. See Appendix F for detailed definitions.)*

F.2 Analysis of interference between primitive vectors

In Table 2 each cell had its own optimized injection magnitude α and injection layer L (maximizing expression for that specific primitive-task combination). To investigate the specificity of algorithmic primitives, we additionally investigated the effect of injecting the algorithmic primitive at the specific α and L that optimizes that primitive's corresponding behavioral hallmark (Table 6).

Primitive vectors were substantially more disentangled in this analysis. The analysis reveals that the entanglement of some primitives seems to follow the sequential order of their usual expression. For instance, injecting `nearest_neighbor` increases the expression of its algorithmic successor, `compute_distance`, but injecting the successor doesn't increase the expression of its predecessor (`nearest_neighbor`). This suggests that the entanglement of two primitives may be, at least partially, due to them being commonly expressed in specific sequential algorithmic motifs, with specific ordering.

	%NN paths ↑	#Paths ↑	Dist. comp. ↓	Final ans. ↓	#Verif. ↑	#Comp. ↑
`nearest_neighbor`	**+56.1%**	-4.2%	-34.9%	+6.3%	+40.2%	-22.0%
`generate_new_path`	-41.4%	**+143.9%**	-76.0%	+85.3%	-93.5%	-82.9%
`compute_distance`	-100.0%	-100.0%	**-86.5%**	+37.9%	+35.9%	+79.3%
`produce_final_answer`	-100.0%	-95.2%	+283.8%	**-81.5%**	-48.9%	-84.1%
`compare_or_verify`	-100.0%	-95.5%	+266.4%	+85.5%	+568.5%	**+1103.7%**
`compare_or_verify_2`	-100.0%	-97.7%	+280.2%	+76.5%	**+706.5%**	+325.6%

Table 6: Effects of primitive vector injection on behavioral hallmarks (% above baseline). For each row, we identify the magnitude and layer maximizing the effect for that row's primitive and then evaluate the effect on all behavioral hallmarks in that row. Bold = strongest effect, underline = effects that increase expression of the behavioral hallmark.

F.3 Reconstructing primitive vectors by their token outputs

To investigate whether primitive vectors can be reconstructed by their token decodings, we follow the approach in Todd et al. (2024): for a given primitive vector v, we apply the LLM's unembedding matrix to generate an output distribution, $Q_t := D(v)$, where D is the unembedding operation. $Q_t \in \mathbb{R}^T$ where T is the number of tokens. We

then truncate this distribution to the top 100 tokens and apply the softmax to generate a probability distribution P_{t100} over the top-100 tokens. We then perform an optimization where $\hat{v}_{t100}$ is inferred as the vector approximating this probability distribution:

$$\hat{v}_{t100} = \arg\min_{v} \text{CE}(v, P_{t100}), \tag{8}$$

where CE is the crossentropy loss. $\hat{v}_{t100}$ thus reflects the information about the primitive vector that is contained within its decoded vocabulary. We then used those inferred vectors to generate new outputs, computing the associated effect on the different behavioral hallmarks. We found that in almost all cases, their effect was smaller than that of the original primitive vector. This indicates that knowledge of the top tokens associated with a given primitive vector are not sufficient to reconstruct its effect and some additional information is needed. A notable exception is given by the impact of `compare_or_verify` on the number of comparisons, for which the inferred vector has an even stronger effect. This suggests that the effect induced by this primitive vector is captured by its top-decoded tokens. Overall, our analysis suggests that primitive vectors usually contain some information beyond what can be inferred from their top decoded tokens.

Metric	PV	v	$\hat{v}_{t100}$
% NN paths ↑	`nearest_neighbor`	**+56.1%**	+21.1%
# Paths ↑	`generate_new_path`	**+143.9%**	+62.0%
Dist. comp. ↓	`compute_distance`	**-86.5%**	-71.9%
Final ans. ↓	`produce_final_answer`	**-81.5%**	-16.2%
# Verif. ↑	`compare_or_verify_2`	**+706.5%**	+71.7%
# Comp. ↑	`compare_or_verify`	+1103.7%	**+2458.5%**

Table: Performance of the primitive vector (PV) (column v) compared to the reconstruction $\hat{v}_{t100}$ that matches the top 100 tokens (both columns show % above baseline). For each primitive vector, we considered the behavioral hallmark on which it had the strongest effect (see Table 2). In all cases except for the number of comparisons, the original PV induces a stronger effect.

G Hyperparameter Comparisons

To evaluate robustness to our chosen hyperparameters, we repeated the clustering analysis for different choices of layer, number of clusters, and number of responses.

G.1 Number of layers

We first repeated the clustering analysis for all layers by increments of five in both Phi-4 and Phi-4-Reasoning when solving Traveling Salesperson (TSP). We then compared the frequency of each layer's clusters showing up both across responses within a model, and across the responses of the two models (the analysis presented in Fig. 4a) (see Fig. 11a). To evaluate uniquely informative patterns across the layers, we computed the correlation across the layer-wise dissimilarity matrices (Fig. 11b). Together, these analyses revealed broad patterns in the effect of layers: after the first layers, there was a large highly correlated cohort in the intermediate layers, especially layers 10-25 (centered on layer 17), and a smaller cohort of highly correlated layers, ranging from 30-40. We observed that intermediate layers captured more aspects of the reasoning hierarchy, as they more strongly reflected the differences between the brute-force and random guess strategies implemented by different responses from Phi-4 (see Appendix D). In contrast, later layers more strongly highlight the differences between Phi-4 and Phi-4-Reasoning. As we are interested in identifying the underlying algorithmic primitives, these findings justify layer 17 as a suitable candidate for clustering.

G.2 Number of clusters

To evaluate robustness to our choice of K, we conducted a similar analysis, this time keeping the layer fixed (17) and varying K by 5, 30, 50, and 100. As described above, we first computed dissimilarity matrices for frequency of cluster occurrences in within-model and cross-model responses per K (Fig. 12a), and then computed the correlation across these matrices (Fig. 12b). Applied to 100 responses to TSP per model (Phi-4 and Phi-4R), this analysis revealed high correlations among K=30, 50, and 100, with the highest Pearson correlation of >0.98 between K=50 and 100.

Next, we computed normalized inertia, or the sum of squared distance between each data point and its assigned cluster centroid for 4 tasks per $K = 5, 10, ..., 100$, with increments of 5 on the x axis (Fig. 12c). We then used the elbow

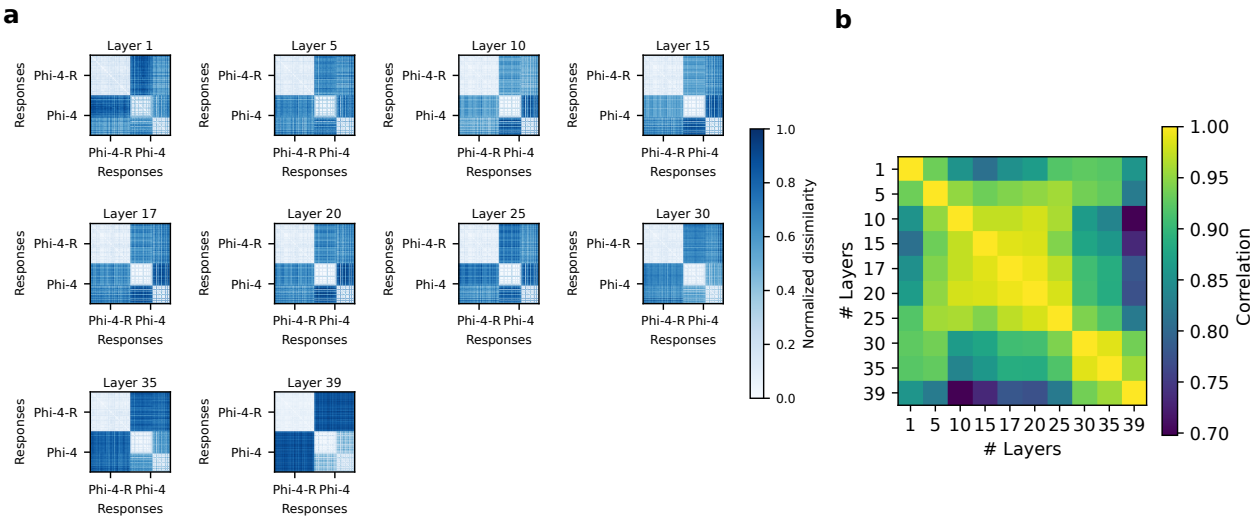

Figure 11: **a** Dissimilarity matrices between the frequency of clusters occurring in different responses, plotted for cluster analyses fitted to different layers. **b** Similarity between these different dissimilarity matrices. **c** Average dissimilarity between responses from different groups: 1) responses by Phi-4-Reasoning, which consistently employ a heuristic nearest-neighbor strategy (see Appendix D), 2) responses by Phi-4 that employ a brute-force strategy, 3) responses by Phi-4 that articulate a random guess. Intermediate layers emphasize similarities between the heuristic strategy employed by Phi-4-Reasoning and both strategies employed by Phi-4, whereas later layers mostly reflect dissimilarities between different models.

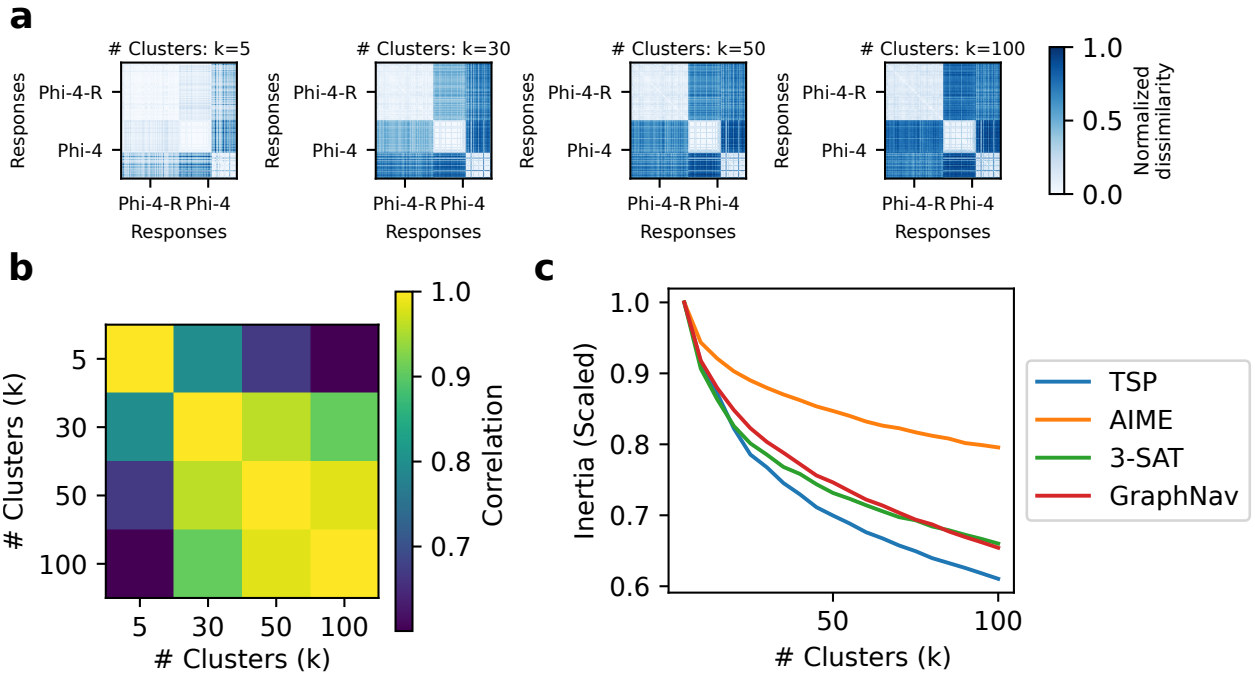

Figure 12: **a** Dissimilarity matrices as in Fig. 4a for different numbers of clusters. **b** Similarity between the dissimilarity matrices for different numbers of clusters, confirming a high degree of similarity for sufficiently many clusters. **c** Inertia (normalized by the inertia for five clusters for each task) plotted against the number of clusters across the five tasks.

method (Han et al., 2012) to identify an approximate K at which inertia dropped for each task. Together, these findings revealed $K = 50$ as the parsimonious and functionally appropriate choice, offering a reasonable tradeoff between explaining variance in the latent space and parsimony in the potential number of algorithmic primitives.